

```
!pip install opencv-python scikit-learn matplotlib numpy
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.13/dist-packages (5.0.0.93)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.13/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (1.6.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cyclor>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (4.64.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (26.3)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (2.9.0.post)
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from joblib>=1.2.0->scikit-learn) (3.0.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

print("Libraries loaded successfully!")
```

Libraries loaded successfully!

```
from google.colab import files
uploaded = files.upload()
```

[Choose Files](#) | images (2).jpg

images (2).jpg(image/jpeg) - 10320 bytes, last modified: 9/15/2026 - 100% done
Saving images (2).jpg to images (2) (2).jpg

```
filename = list(uploaded.keys())[0]

img = cv2.imread(filename)

# OpenCV BGR ko RGB mein convert karna
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

print("Image loaded successfully!")
print("Image size:", img.shape)
```

Image loaded successfully!
Image size: (280, 180, 3)

```
plt.figure(figsize=(8, 10))

plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.show()
```

Original Image



```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(8, 10))

plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.show()
```

Grayscale Image



```
# Gaussian Blur
blur = cv2.GaussianBlur(gray, (15, 15), 0)

plt.figure(figsize=(14, 7))

plt.subplot(1, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")

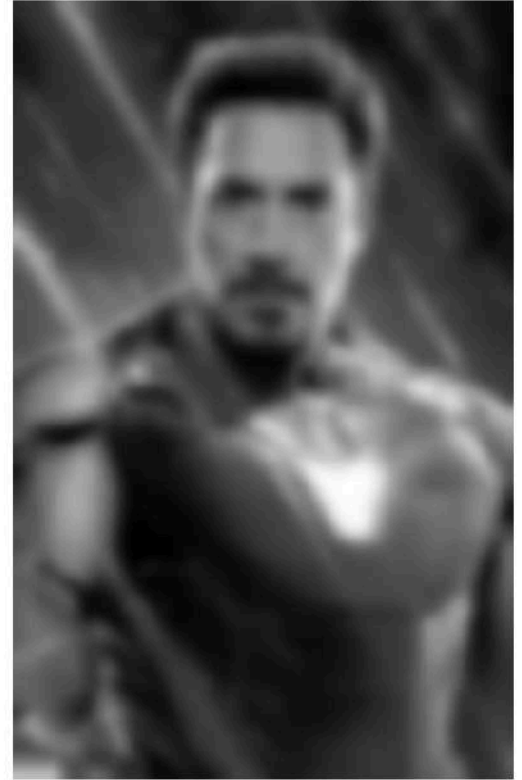
plt.subplot(1, 2, 2)
plt.imshow(blur, cmap="gray")
plt.title("Gaussian Blurred Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Grayscale Image



Gaussian Blurred Image



```
ret, global_thresh = cv2.threshold(  
    blur,  
    127,  
    255,  
    cv2.THRESH_BINARY  
)  
  
plt.figure(figsize=(8, 10))  
  
plt.imshow(global_thresh, cmap="gray")  
plt.title("Global Thresholding")  
plt.axis("off")  
  
plt.show()  
  
print("Threshold value used:", ret)
```

Global Thresholding



Threshold value used: 127.0

```
otsu_value, otsu_thresh = cv2.threshold(
    blur,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
plt.figure(figsize=(8, 10))

plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu's Thresholding")
plt.axis("off")

plt.show()

print("Automatically selected Otsu threshold:", otsu_value)
```

Otsu's Thresholding



Automatically selected Otsu threshold: 92.0

```
adaptive_thresh = cv2. adaptiveThreshold(  
    blur,  
    255,  
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
    cv2. THRESH_BINARY,  
    11,  
    2  
)  
plt.figure(figsize=(8, 10))  
  
plt.imshow(adaptive_thresh, cmap="gray")  
plt. title("Adaptive Thresholding")  
plt.axis("off")  
  
plt.show()
```

Adaptive Thresholding



```
# Binary image
_, thresh = cv2.threshold(
    gray,
    0,
    255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

# Noise removal
kernel = np.ones((3, 3), np.uint8)

opening = cv2.morphologyEx(
    thresh,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
)

# Background
sure_bg = cv2.dilate(
    opening,
    kernel,
    iterations=3
)

# Distance transform
dist_transform = cv2.distanceTransform(
    opening,
    cv2.DIST_L2,
    5
)

# Foreground
_, sure_fg = cv2.threshold(
    dist_transform,
    0.5 * dist_transform.max(),
    255,
```

```
0
)

sure_fg = np.uint8(sure_fg)

# Unknown region
unknown = cv2.subtract(
    sure_bg,
    sure_fg
)

# Marker labelling
num_labels, markers = cv2.connectedComponents(sure_fg)

markers = markers + 1

markers[unknown == 255] = 0

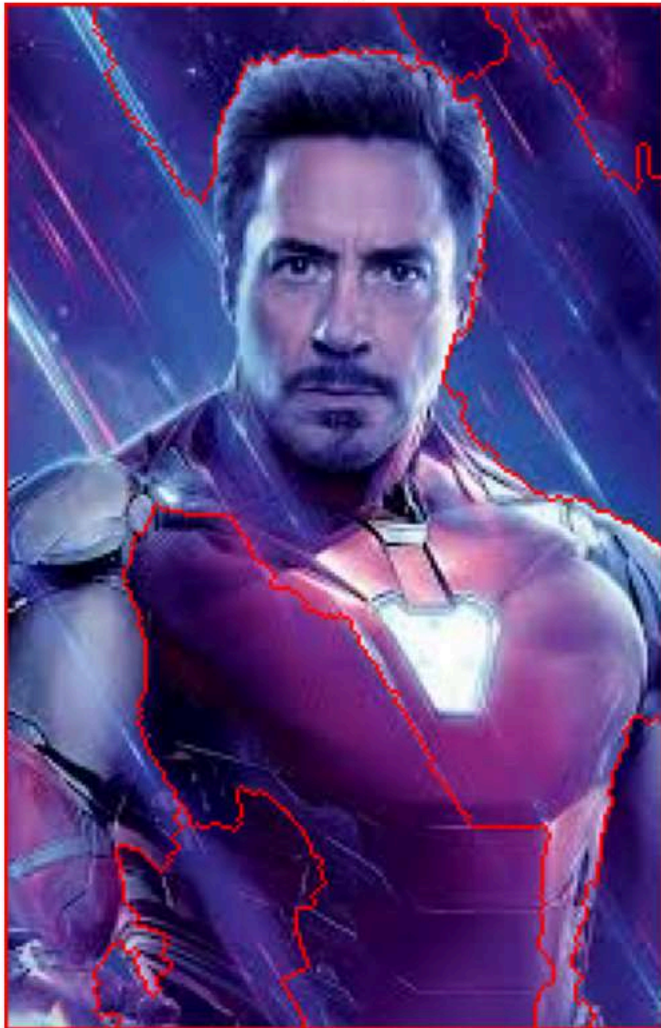
# Apply watershed
watershed_img = img_rgb.copy()

markers = cv2.watershed(
    cv2.cvtColor(watershed_img, cv2.COLOR_RGB2BGR),
    markers
)

# Mark watershed boundaries in red
watershed_img[markers == -1] = [255, 0, 0]

# Display result
plt.figure(figsize=(8, 10))
plt.imshow(watershed_img)
plt.title("Watershed Segmentation")
plt.axis("off")
plt.show()
plt.show()
```

Watershed Segmentation



```

# Convert image into pixels
pixel_data = img_rgb.reshape((-1, 3))
# Convert to float
pixel_data = np.float32(pixel_data)
# Number of clusters
K = 3

# K-Means model
kmeans = KMeans(
    n_clusters=K,
    random_state=42,
    n_init=10
)

labels = kmeans.fit_predict(pixel_data)
# Cluster centers
centers = np.uint8(kmeans.cluster_centers_)
# Replace each pixel with its cluster center
segmented_data = centers[labels]
# Reshape back to original shape
segmented_image = segmented_data.reshape(img_rgb.shape)
plt.figure(figsize=(8, 10))
plt.imshow(segmented_image)
plt.title("K-Means Segmentation (K=3)")
plt.axis("off")
plt.show()

```

K-Means Segmentation (K=3)



```

plt.figure(figsize=(16, 12))

plt.subplot(2, 3, 1)
plt.imshow(img_rgb)
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(global_thresh, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")

```

```
plt.subplot(2, 3, 3)
plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu Thresholding")
plt.axis("off")
```

```
(np.float64(-0.5), np.float64(179.5), np.float64(279.5), np.float64(-0.5))
```

Original



Global Thresholding



Otsu Thresholding



```
plt.figure(figsize=(16, 12))
plt.subplot(2, 3, 4)
plt.imshow(adaptive_thresh, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")
```

```
plt.subplot(2, 3, 5)
plt.imshow(watershed_img)
plt.title("Watershed")
plt.axis("off")
```

```
plt.subplot(2, 3, 6)
plt.imshow(segmented_image)
plt.title("K-Means (K=3)")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

Adaptive Thresholding



Watershed



K-Means (K=3)



```
plt.figure(figsize=(6, 9))

plt.imshow(img_rgb)

plt.axis("off")

plt.tight_layout(pad=0)

plt.savefig(
    "original_image.png",
    dpi=300,
    bbox_inches="tight",
    pad_inches=0
)

plt.show()
```

